KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Öğrenci Ders Notu

HAFTA 06

Statik ve Nesne Üyesi Değişkenler

`static` alan ve metotlar · sınıf düzeyi ile nesne düzeyi · sınıf yükleme ve `static` bloklar · `static final` sabitler · sayaç deseni ve yardımcı sınıflar

ÖZET

Hafta 5'te iki yerde `static` anahtar sözcüğünü kullanmak zorunda kaldık ama açıklamasını erteledik: statik fabrika metotlarında ve `Bilet` sınıfının otomatik numara üreten sayacında. Bu hafta o borcu ödüyoruz. Bir alanın **nesneye mi yoksa sınıfa mı** ait olduğu sorusu, ilk bakışta küçük bir sözdizimi ayrıntısı gibi görünür; oysa yanlış verilmiş bu karar, birbirinden bağımsız olması gereken nesneleri sessizce birbirine bağlar ve derleyicinin hiçbir uyarı vermediği hatalar üretir. Haftanın sonunda `static` alan ile örnek alanın bellekteki karşılığını, sınıf yükleme zamanlamasını, `static` blokların ne zaman çalıştığını, `static final` sabitlerin sınırlarını ve yardımcı (utility) sınıf yazımını öğrenmiş olacaksınız. Bölüm 4.6'da, Hafta 14'te ayrıntılı göreceğimiz **tekil (singleton)** deseninin sezgisine de kısa bir bakış yapılacaktır.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

İçindekiler

1. Bu Haftanın Yol Haritası ... 3

1.1. Ön Koşul Bilgi Kontrolü ... 3

1.2. Haftanın Zaman Planı ... 4

2. Motivasyon: Numarayı Kim Vermeli? ... 5

2.1. Sorunu Üreten Kod ... 5

2.2. Aynı Sınıf, Sınıf Düzeyi Bir Alanla ... 5

2.3. Kazancın Ölçüsü ... 7

3. Kavramın Tanımı ve Sözdizimi ... 9

3.1. Bellekte Ne Değişiyor? ... 9

3.2. Sözdizimi: `static` Nereye Yazılır? ... 10

3.3. Erişim: `static` Metot Neden Örnek Alanı Göremez? ... 11

3.4. Çağırma Biçimi: `Sinif.uye` mi, `nesne.uye` mi? ... 12

4. Derinleşme ... 14

4.1. Sınıf Yükleme ve `static` Bloklar ... 14

4.2. `static` Bloğun Asıl İşi: Kurulum Doğrulaması ... 16

4.3. Yardımcı (Utility) Sınıflar ... 17

4.4. `static final` Sabitler ve Sihirli Sayılar ... 19

4.5. `final`'ın Sınırı: Referansı Korur, İçeriği Korumaz ... 20

4.6. Tekil (Singleton) Sezgisi ... 21

4.7. `static` mi, Örnek Üyesi mi? Karar Akışı ... 23

5. C'den Java'ya Köprü ... 25

6. Sık Yapılan Hatalar ... 27

6.1. `main` İçinden Örnek Üyeye Erişmek ... 27

6.2. Nesneye Özgü Alanı `static` Yapmak ... 28

6.3. Sabiti Değiştirilebilir Bırakmak ... 28

6.4. Her Şeyi `static` Yaparak Nesnelerden Kaçmak ... 29

6.5. `static` Alanı Nesne Üzerinden Çağırmak ... 29

6.6. `static` Bloğu Sıraya Dikkat Etmeden Yazmak ... 29

6.7. Derleyici İletileri Sözlüğü ... 29

7. İyi Tasarım İlkeleri ... 31

7.1. Adlandırma ve Yazım Kuralları ... 31

8. Bir Sonraki Haftaya Köprü ... 32

9. Haftanın Özeti ... 32

10. Alıştırmalar ... 33

10.1. Kavram Soruları ... 33

10.2. Kod Okuma ve Hata Bulma ... 34

10.3. Programlama Alıştırmaları ... 35

11. Öz Değerlendirme ... 37

Öz Değerlendirme Kontrol Listesi 37
12. Kaynaklar 37
12.1. Ders Kitapları 37
12.2. Çevrimiçi Kaynaklar 38
12.3. İletişim ve Destek 38

1. Bu Haftanın Yol Haritası

Hafta 2'den bu yana yazdığımız her sınıfta alanlar **nesneye** aitti. `Ogrenci` sınıfının `ad` alanı, her `new Ogrenci(...)` çağrısıyla yeniden doğuyor ve o nesneye özel bir değer tutuyordu. Hafta 4'te bu alanları `private` yapıp kapsülledik; Hafta 5'te ise doğdukları anı yapıcı zinciriyle denetim altına aldık. Üç haftadır aynı varsayımla çalışıyoruz: *bir alan bir nesneye aittir*.

Bu varsayım her zaman doğru değildir. Bazı bilgiler tek tek nesnelere değil, **sınıfın tamamına** aittir: "Şu ana kadar kaç öğrenci nesnesi üretildi?", "Bir sonraki öğrenci numarası kaç olmalı?", "Asgari kayıt yaşı nedir?" Bu üç soru da belirli bir öğrenciyle ilgili değildir; `Ogrenci` kavramının kendisiyle ilgilidir. Java'da bu ayrımı yapan anahtar sözcük `static`'tir.

Bu haftanın teknik yükü hafiftir: öğrenilecek tek bir anahtar sözcük vardır. Asıl zorluk, Hafta 4 ve 5'te olduğu gibi **karar vermededir**. Bir üyenin `static` mi yoksa örnek üyesi mi olacağına verilen karar, sınıfın davranışını kökten değiştirir; üstelik yanlış karar çoğu zaman derleme hatası üretmez, sessizce yanlış çalışan bir program üretir.

◎ Bu Bölümün Sonunda Öğrenci

- `static` alan ile örnek alanın bellekteki karşılığını çizebilir; kaç nesne üretilirse üretilsin `static` alandan kaç kopya bulunduğunu söyleyebilir.
- Bir üyenin `static` mi örnek üyesi mi olması gerektiğine **gerekçeli** karar verebilir; kararını "bu değer tüm nesneler için aynı mı?" sorusuna dayandırabilir.
- `static` metodun neden örnek alanlara erişemediğini açıklayabilir; `Cannot make a static reference to the non-static field` hatasını okuyup düzeltebilir.
- Sınıf yüklemenin **tembel** olduğunu ve `static` blokların sınıf başına yalnızca bir kez çalıştığını çalışan bir çıktı üzerinde gösterebilir.
- `static final` sabitler tanımlayabilir; sihirli sayıları (magic number) adlandırılmış sabitlere dönüştürebilir.
- `final` anahtar sözcüğünün referans türlerde neyi koruyup neyi korumadığını açıklayabilir ve savunmalı kopya (defensive copy) yazabilir.
- Otomatik kimlik üreten **sayaç desenini** uygulayabilir; benzersizlik güvencesini çağıran koddan sınıfa taşıyabilir.
- Üç kurala uyan bir **yardımcı (utility) sınıf** yazabilir ve bu sınıfın ne zaman uygun, ne zaman yanlış bir tercih olduğunu değerlendirebilir.

1.1. Ön Koşul Bilgi Kontrolü

Aşağıdaki soruların tamamını Hafta 1–5 bilgisiyle yanıtlayabiliyor olmalısınız. Yanıtlayamadığınız madde varsa laboratuvar öncesinde ilgili bölümü tekrar etmeniz önerilir.

1. `new Ogrenci("Elif", 20)` çağrısı çalıştığında bellekte tam olarak ne olur? Alanlar nerede saklanır? (Hafta 5, Bölüm 3.1)
2. Bir nesnenin alanlarına `this` üzerinden erişiyoruz. `this` neyi işaret eder ve hangi bağlamlarda kullanılamaz?

3. Hafta 5'te öğrendiğimiz nesne başlatma sırası neydi? Yapıcı gövdesi bu sıranın neresinde çalışır?
4. `final` bir alan kaç kez atanabilir? `private final int[] notlar;` bildirimi dizinin **içeriğini** korur mu? (Hafta 5, Bölüm 4.2)
5. Bir metodun `private` olması ne anlama gelir? Hafta 4'te erişim belirleyicilerinin dört düzeyi nelerdi?
6. Hafta 5'te `Kayit` sınıfında `public static final Kayit BOS` bildirimini görmüştük. O satırdaki üç anahtar sözcüğün her biri ne yapıyordu? Hangisini henüz açıklamamıştık?

1.2. Haftanın Zaman Planı

Oturum	Süre	İçerik	Çıktı
Teori 1	45 dk	Motivasyon · sınıf düzeyi / nesne düzeyi ayrımı · bellek şeması · erişim kuralları	`static` / örnek kararını gerekçeli verebilme
Teori 2	45 dk	Sınıf yükleme ve `static` bloklar · `static final` sabitler · sayaç deseni · yardımcı sınıflar · tekil sezgisi	Sayaç ve yardımcı sınıf yazabilme
Lab 1	60 dk	Görev 0–2: ortam kontrolü · otomatik numara üreten sayaç · yardımcı sınıf	Çalışan ve doğrulayan iki sınıf
Lab 2	60 dk	Görev 3–5: `static final` sabitler ve savunmalı kopya · hata ayıklama · (bonus) tekil kayıtçı	Teslim edilecek `.zip` paketi

Tablo 6.1 — Hafta 6'nın oturum planı ve beklenen çıktılar

> **ⓘ BİLGİ**
>
> Bu hafta öğrenilecek yeni anahtar sözcük **tek tanedir**: `static`. Ancak bu tek sözcük dört ayrı yerde kullanılır — alan, metot, blok ve (Hafta 10'da göreceğimiz) iç sınıf bildiriminde. Dördünde de anlam aynıdır: *"bu üye bir nesneye değil, sınıfın kendisine aittir."*

2. Motivasyon: Numarayı Kim Vermeli?

Bir kütüphane yazılımı yazıyoruz. Her kitabın benzersiz bir demirbaş numarası olmalı: `KTP-00001`, `KTP-00002`, ... Şu ana kadar öğrendiklerimizle bu numarayı yapıcıya parametre olarak vermek en doğal çözüm gibi görünür.

2.1. Sorunu Üreten Kod

Kitap.java (sorunlu sürüm)

```java
// SORUNLU TASARIM - benzersizlik CAGIRAN kodun sorumlulugunda
public class Kitap {
    private final String demirbasNo;
    private final String baslik;

    public Kitap(String demirbasNo, String baslik) {
        this.demirbasNo = demirbasNo;
        this.baslik = baslik;
    }
}

// Kullanan kod:
new Kitap("KTP-00001", "Effective Java");
new Kitap("KTP-00002", "Clean Code");
new Kitap("KTP-00002", "Core Java");     // CAKISMA - kimse fark etmez
```

Kod 6.1 — Numarayı çağıran koda bırakan tasarım

Bu kodda üç ayrı kusur vardır ve üçü de aynı kökten çıkar:

1. **Benzersizlik güvencesi sınıfın dışındadır.** `Kitap` sınıfı kendi en temel kuralını (“iki kitabın numarası aynı olamaz”) uygulayamaz; kuralı uygulaması beklenen taraf, sınıfı kullanan koddur.
2. **Çakışma hiçbir aşamada yakalanmaz.** Derleyici susar, çalışma zamanında istisna oluşmaz. Hata, aylar sonra iki kitabın aynı raftaymış gibi göründüğü bir raporda ortaya çıkar.
3. **Sıradaki numaranın kaç olduğunu kimse bilmez.** Bu bilgi hiçbir nesnede saklanamaz; çünkü tek bir kitaba ait bir bilgi değildir.

> ✖ **ÇÖZÜMSÜZLÜK — bilginin ait olduğu yer yok**
>
> Üçüncü madde bu haftanın anahtarıdır. “Sıradaki numara” bilgisi hangi nesneye konabilir? Birinci kitaba mı? Sonuncusuna mı? Hiçbirine konamaz, çünkü bu bilgi **tek bir kitabın değil, kitapların tamamının** bilgisidir. Şu ana kadar öğrendiğimiz araçlarla saklanacak bir yeri yoktur.

2.2. Aynı Sınıf, Sınıf Düzeyi Bir Alanla

Java'da bir alanın önüne `static` yazıldığında o alan artık nesnelere değil, **sınıfın kendisine** ait olur. Kaç nesne üretilirse üretilsin bellekte o alandan yalnızca bir tane bulunur. Sıradaki numarayı tam olarak burada saklarız:

```java
// ---- SINIF DUZEYI ----
private static final String ON_EK = "KTP-";
private static final int    BASLANGIC_NO = 1;
public  static final int    AZAMI_ODUNC_GUN = 21;

private static int sonrakiNo  = BASLANGIC_NO;
private static int uretilen   = 0;
private static int oduncteki  = 0;
```

Kod 6.2 — Sınıf düzeyi alanlar: sayaç ve sabitler

Listenin **2–4. satırları** bu haftanın ikinci yeni kavramını da getiriyor: `static final` **sabitler**. Bunlara Bölüm 4.4'te döneceğiz. Şimdilik **6–8. satırlara** odaklanalım: `sonrakiNo`, `uretilen` ve `oduncteki` alanları `static`'tir; yani bellekte her birinden yalnızca bir kopya vardır ve tüm `Kitap` nesneleri aynı kopyayı paylaşır.

```java
public Kitap(String baslik, String yazar) {
    if (baslik == null || baslik.isBlank()) {
        throw new IllegalArgumentException("Baslik bos olamaz.");
    }
    if (yazar == null || yazar.isBlank()) {
        throw new IllegalArgumentException("Yazar bos olamaz.");
    }
    this.demirbasNo = numaraUret();   // sinif ureteci cagriliyor
    this.baslik = baslik;
    this.yazar  = yazar;
    this.odunctemi = false;
    uretilen++;
}

// Numara uretimi SINIFIN isidir; bir nesneye ait degildir -> static
private static String numaraUret() {
    String s = String.format("%s%05d", ON_EK, sonrakiNo);
    sonrakiNo++;
    return s;
}
```

*Kod 6.3 — Yapıcı numarayı istemez, sınıftan **ister***

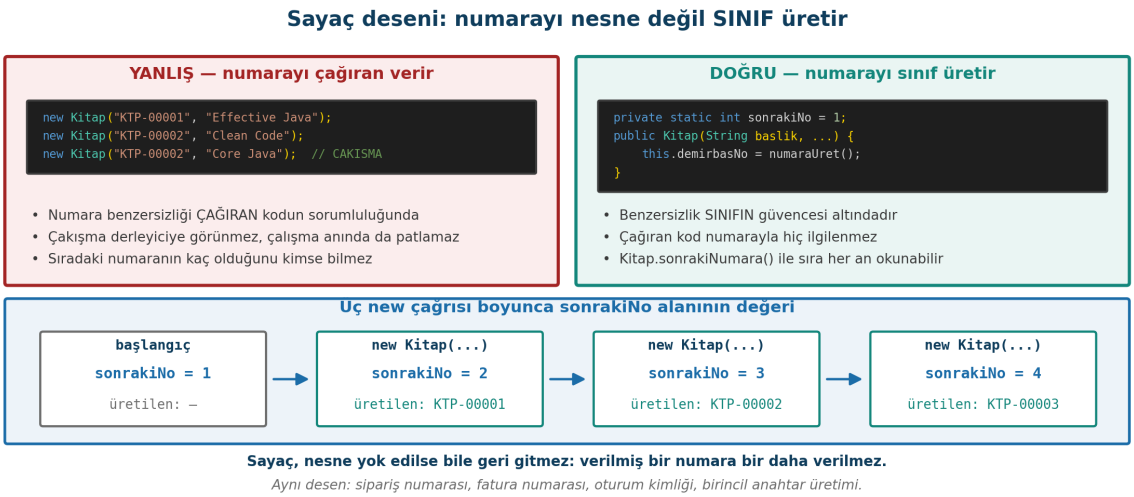
Yapıcının imzasında artık `demirbasNo` parametresi yoktur; çağıran kod numarayla hiç ilgilenmez. **8. satırda** yapıcı, numarayı sınıfın kendi üreticisinden ister. **16–20. satırlardaki** `numaraUret()` metodu da `static`'tir: bir kitabın değil, sınıfın işidir.

```
▸ TERMINAL — java Gorev1_KitapDemo (otomatik numaralar)
=== 1) Hic kitap yokken sinif durumu ===
  uretilenSayisi = 0
  sonrakiNumara  = KTP-00001

=== 2) Bes kitap uretiliyor ===
  KTP-00001 | Nesne Yonelimli Programlama / Deitel | RAFTA
  KTP-00002 | Core Java Volume I / Horstmann | RAFTA
  KTP-00003 | Effective Java / Bloch | RAFTA
  KTP-00004 | Java: The Complete Reference / Schildt | RAFTA
```

```
KTP-00005 | Clean Code / Martin | RAFTA
Numaralar KESINTISIZ ve BENZERSIZ verildi.
```

Beş kitap üretildi ve beşi de kesintisiz, benzersiz numara aldı. Üstelik ilk iki satıra dikkat ediniz: `uretilenSayisi` ve `sonrakiNumara` değerleri **hiçbir nesne üretilmeden önce** okunabildi. Bu, örnek alanlarla imkânsızdır; okunacak bir nesne yoktu.



Şekil 6.1 — Sayaç deseni: benzersizlik güvencesi çağıran koddan sınıfa taşınır

2.3. Kazancın Ölçüsü

Ölçüt	Numarayı çağıran verir	Numarayı sınıf üretir
Benzersizlik güvencesi	Çağıran kodda	**Sınıfın kendisinde**
Çakışma olasılığı	Her çağrıda var	Yok
Çakışmanın yakalanması	Hiçbir aşamada	Kavram olarak oluşmaz
Sıradaki numaranın okunması	Mümkün değil	`Kitap.sonrakiNumara()`
Yapıcı imzası	`Kitap(String, String)`	`Kitap(String, String)` — ama anlamlı iki alan
Numara biçimi değişirse	Tüm çağrı yerleri	Tek metot: `numaraUret()`
Üretilen toplam sayı	Ayrıca sayılmalı	Zaten `uretilen` alanında

Tablo 6.2 — Sayacı sınıfa taşımanın somut sonuçları

> ✓ **İPUCU**
>
> Bu desen yazılımın her yerindedir: sipariş numarası, fatura numarası, oturum kimliği, veritabanı birincil anahtarı, günlük kaydı sıra numarası. Ortak nokta hep aynıdır — üretilecek değerin, **var olan hiçbir nesneye ait olmayan** bir durumdan türetilmesi gerekir.

> ▲ **DİKKAT**

> Sayaç bir nesne yok edildiğinde **geri gitmez**. `uretilen` alanı “şu anda kaç kitap var” sorusunu değil, “bugüne kadar kaç kitap üretildi” sorusunu yanıtlar. Verilmiş bir numara bir daha verilmez; benzersizliğin bedeli budur.

3. Kavramın Tanımı ve Sözdizimi

`static` üye (sınıf üyesi / class member) Bir nesneye değil, **sınıfın kendisine** ait olan alan, metot ya da blok. Sınıf yüklendiğinde bir kez bellekte yerini alır ve programın sonuna kadar orada kalır. Kaç nesne üretilirse üretilsin bellekte **tek kopyası** bulunur.

Örnek üyesi (instance member) Bir nesneye ait olan alan ya da metot. Her `new` çağrısıyla yeni bir kopyası doğar; nesne ulaşılamaz hâle geldiğinde çöp toplayıcıyla birlikte yok olur.

Sınıf alanı (metot alanı / method area) JVM'in `static` alanları sakladığı bellek bölgesi. Öbekten (heap) ayrıdır ve sınıf yüklendiğinde ayrılır.

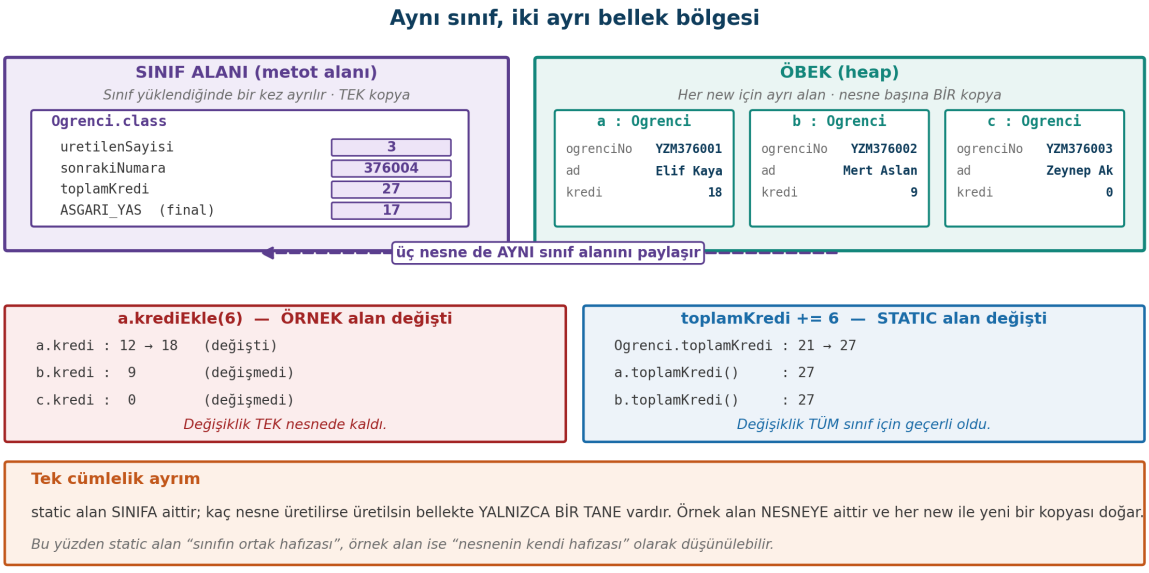
`static` blok (statik başlatıcı / static initializer) Sınıf yüklenirken **bir kez** çalışan, ad taşımayan `static { ... }` bloğu. Tek satırlık bir atamayla kurulamayacak sabitleri hazırlamak ve kurulum tutarlılığını doğrulamak için kullanılır.

`static final` sabit Sınıfa ait olan ve bir kez atandıktan sonra değiştirilemeyen alan. Java geleneğinde SCREAMING_SNAKE_CASE ile adlandırılır.

Yardımcı sınıf (utility / helper class) Hiç örnek üyesi bulunmayan, yalnızca `static` metot ve sabit barındıran sınıf. `java.lang.Math` bu türün en bilinen örneğidir.

3.1. Bellekte Ne Değişiyor?

`static` anahtar sözcüğünün yaptığı tek şey, alanın **nerede** saklanacağını değiştirmektir. Örnek alanlar öbekte, her nesnenin kendi bölgesinde durur; `static` alanlar ise sınıf alanında, tek bir kopya hâlinde bulunur.



Şekil 6.2 — Sınıf alanı ile öbek: tek kopya ve nesne başına kopya

Şekildeki iki bölge arasındaki fark, dersin geri kalanının tamamını açıklar. Üç `Ogrenci` nesnesinin her birinin kendi `kredi` alanı vardır; ama üçü de aynı `toplamKredi` alanını gösterir. Bu davranışı çalışan bir programla doğrulayalım:

```java
// ---- SINIF DUZEYI (static) : tum nesnelerce paylasilan TEK kopya ----
private static int uretilenSayisi = 0;      // kac nesne uretildi?
private static int sonrakiNumara  = 376001; // bir sonraki ogrenci numarasi
private static int toplamKredi    = 0;      // butun ogrencilerin kredi toplami

// ---- SINIF DUZEYI SABIT (static final) : degismez, tek kopya ----
public  static final int    ASGARI_YAS   = 17;
public  static final int    AZAMI_KREDI  = 45;
public  static final String BOLUM_KODU   = "YZM";

// ---- NESNE DUZEYI (ornek/instance) : her nesnede AYRI kopya ----
private final String ogrenciNo;
private final String ad;
private int yas;
private int kredi;
```

Kod 6.4 — `Ogrenci` sınıfının alan bildirimleri: üç grup

Alanlar bilinçli olarak üç gruba ayrılmış ve her grubun başına bir yorum satırı konmuştur. Bu, bu haftadan itibaren yazacağınız her sınıfta uygulamanız beklenen bir düzendir: okuyucu, sınıfın hangi durumunun paylaşıldığını ilk on satırda görmelidir.

```
▸ TERMINAL — java OgrenciDemo (örnek ve static alanın karşılaştırması)
=== 3) ORNEK alan degisirse yalnizca O nesne degisir ===
  a.krediEkle(6) cagriliyor...
  a.getKredi() = 18   (degisti)
  b.getKredi() = 9    (degismedi)
  c.getKredi() = 0    (degismedi)

=== 4) STATIC alan degisirse HERKES icin degisir ===
  toplamKredi   = 27
  ortalamaKredi = 9.0
  a uzerinden okundu : 27
  b uzerinden okundu : 27
  Ayni sayi cikti; cunku ortada TEK bir toplamKredi var.
```

Çıktının anlattığı iki cümle şudur. Birincisi: `a.krediEkle(6)` çağrısı yalnızca `a` nesnesinin `kredi` alanını değiştirdi; `b` ve `c` etkilenmedi. İkincisi: aynı çağrı `toplamKredi` alanını da artırdı ve bu değişiklik `a` üzerinden de `b` üzerinden de aynı görülüyor — çünkü ortada **tek bir** `toplamKredi` vardır.

3.2. Sözdizimi: `static` Nereye Yazılır?

Kullanım	Sözdizimi	Anlamı	Kaç kopya / kaç kez?
Sınıf alanı	`static int uretilen;`	Alan sınıfa aittir	Sınıf başına 1 kopya
Sınıf sabiti	`static final int GECME = 50;`	Değişmeyen sınıf alanı	Sınıf başına 1 kopya
Sınıf metodu	`static int kare(int x)`	Metot nesne gerektirmez	—
Sınıf kurulum bloğu	`static { ... }`	Sınıf yüklenirken çalışır	Sınıf başına 1 kez
Örnek alanı	`int kredi;`	Alan nesneye aittir	Nesne başına 1 kopya

Kullanım	Sözdizimi	Anlamı	Kaç kopya / kaç kez?
Örnek metodu	`int getKredi()`	Metot `this` üzerinden çalışır	—
Örnek bloğu	`{ ... }`	Her `new` için çalışır (Hafta 5)	Nesne başına 1 kez

Tablo 6.3 — `static` anahtar sözcüğünün kullanıldığı yerler

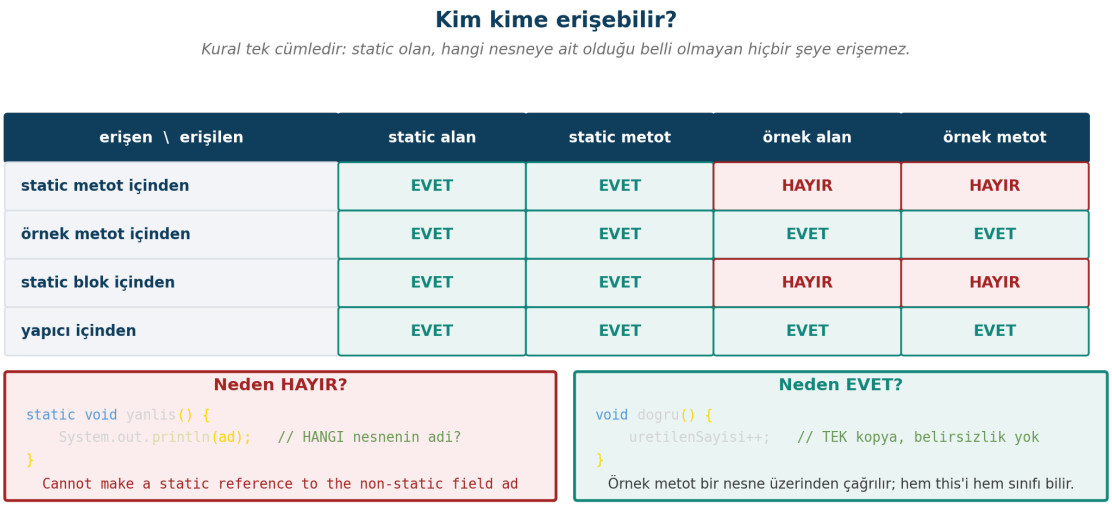
`static` sözcüğü erişim belirleyicisinden **sonra**, tür adından **önce** yazılır. Java geleneğinde sıralama şöyledir: erişim belirleyicisi → `static` → `final` → tür → ad.

```
1  public static final int AZAMI_KREDI = 45;
2  //  (1)   (2)    (3)  (4)     (5)
3  // (1) erisim : disaridan gorunur mu?
4  // (2) static : nesneye mi sinifa mi ait?
5  // (3) final  : bir daha atanabilir mi?
6  // (4) tur    : hangi turden deger tutar?
7  // (5) ad     : SCREAMING_SNAKE_CASE (sabit oldugu icin)
```

Kod 6.5 — Bir sabit bildiriminin beş parçası

3.3. Erişim: `static` Metot Neden Örnek Alanı Göremez?

Bu haftanın en sık karşılaşılan derleme hatası, `static` bir metodun içinden bir örnek alanına erişmeye çalışmaktır. Nedeni tek bir soruyla anlaşılır: **hangi nesnenin alanı?**



Şekil 6.3 — `static` üyeler, hangi nesneye ait olduğu belirsiz hiçbir şeye erişemez

Ogrenci.java

```
1      // ---- SINIF METOTLARI (static) : nesne gerektirmez ----
2      public static int uretilenSayisi()  { return uretilenSayisi; }
3      public static int sonrakiNumara()   { return sonrakiNumara; }
4      public static int toplamKredi()     { return toplamKredi; }
5
6      public static double ortalamaKredi() {
7          if (uretilenSayisi == 0) {
8              return 0.0;    // sifira bolme korumasi
9          }
10         return (double) toplamKredi / uretilenSayisi;
```

```
11      }
12
13      /* Static metot ORNEK alana erisemez; cunku hangi nesnenin alani
14       * oldugu belirsizdir. Asagidaki satir derleme hatasi verirdi:
15       *     public static void yanlis() { System.out.println(ad); }
16       *     -> Cannot make a static reference to the non-static field ad
17       */
```

Kod 6.6 — `static` metotlar yalnızca `static` alanları okuyabilir

6–11. satırlardaki `ortalamaKredi()` metodu iki `static` alanı okur ve bunda bir belirsizlik yoktur: her ikisinden de tek kopya vardır. **13–16. satırlardaki** yorum bloğu ise yasak olan kullanımı ve derleyicinin vereceği iletiyi göstermektedir.

Erişen	`static` alan	`static` metot	Örnek alan	Örnek metot
`static` metot içinden	Evet	Evet	**Hayır**	**Hayır**
`static` blok içinden	Evet	Evet	**Hayır**	**Hayır**
Örnek metodu içinden	Evet	Evet	Evet	Evet
Yapıcı içinden	Evet	Evet	Evet	Evet

Tablo 6.4 — `static` ve örnek üyeler arasındaki erişim kuralları

Tablonun okunuşu tek yönlüdür ve bu asimetri önemlidir: örnek metotlar `static` üyelere erişebilir, ama `static` metotlar örnek üyelere erişemez. Gerekçe basittir — bir örnek metodu her zaman bir nesne üzerinden çağrılır ve dolayısıyla hem `this`'i hem sınıfı bilir; `static` bir metodun ise `this`'i yoktur.

> ✖ **EN SIK HATA — `main` de bir `static` metottur**
>
> `public static void main(String[] args)` metodu da bir `static` metottur. Bu yüzden `main` içinden doğrudan bir örnek alanına erişemezsiniz. Java'ya yeni başlayanların en sık aldığı derleme hatası budur. Çözüm, `main` içinde önce bir nesne üretip alana o nesne üzerinden erişmektir.

3.4. Çağırma Biçimi: `Sinif.uye` mi, `nesne.uye` mi?

Bir `static` üyeye iki biçimde erişilebilir. Java her ikisine de izin verir; ancak ikisi **aynı derecede doğru değildir**.

StatikCozumleme.java

```
1       static String kim() { return "STATIK metot"; }
2       String kimOrnek()   { return "ORNEK metot";  }
```

Kod 6.7 — Karşılaştırma için bir `static` bir de örnek metodu

```
▸ TERMINAL — java StatikCozumleme (çözümleme biçimleri)
=== 3) Nesne uzerinden static cagirmak ===
  StatikCozumleme.kim() = STATIK metot
  n.kim()               = STATIK metot   <- derlenir ama YANILTICI
  n.kimOrnek()          = ORNEK metot
```

```
=== 4) null referans uzerinden static erisim ===
  bos = null;  bos.kim() = STATIK metot
  NullPointerException OLMADI: cagri derleme aninda
  StatikCozumleme.kim() bicimine cevrilmisti.
```

Çıktının son bölümü çarpıcıdır: `bos` değişkeni `null` olmasına rağmen `bos.kim()` çağrısı `NullPointerException` fırlatmadı. Bunun nedeni, derleyicinin bu çağrıyı daha derleme anında `StatikCozumleme.kim()` biçimine çevirmiş olmasıdır; nesneye hiç bakılmaz. İşte bu yüzden `nesne.staticUye` yazımı **yanıltıcıdır**: okuyucuya nesneyle ilgili bir iş yapılıyormuş izlenimi verir.

> ✓ **İPUCU**
>
> Kural: `static` üyelere **her zaman sınıf adı üzerinden** erişin — `Kitap.uretilenSayisi()`, `Math.max(a, b)`, `Integer.MAX_VALUE`. Modern IDE'ler `nesne.staticUye` yazımını sarı uyarıyla işaretler; bu uyarıyı görmezden gelmeyiniz.

4. Derinleşme

4.1. Sınıf Yükleme ve `static` Bloklar

Hafta 5'te bir **nesnenin** doğuş sırasını incelemiştik. Şimdi bir basamak yukarı çıkıyoruz: bir **sınıf** ne zaman belleğe alınır?

Yanıt çoğu öğrenciyi şaşırtır: sınıf, program başlarken değil, **ilk kez gerçekten kullanıldığında** yüklenir. Buna *tembel yükleme* (lazy loading) denir. Sınıf yüklendiğinde önce `static` alan başlatıcıları, sonra `static` bloklar — kaynak koddaki yazım sırasına göre — çalışır ve bu **sınıf başına yalnızca bir kez** olur.



Şekil 6.4 — Sınıf yükleme zamanlaması: `static` bölüm bir kez, örnek bölüm her `new` için

StatikBlokOrnegi.java

```
1  class Oturum {
2      // --- sinif duzeyi ---
3      static int acilanToplam = 0;
4      static final int AZAMI_ESZAMANLI = 3;
5      static {
6          System.out.println("      [O-S1] Oturum static blogu calisti "
7                             + "(AZAMI_ESZAMANLI=" + AZAMI_ESZAMANLI + ")");
8      }
9
10     // --- nesne duzeyi ---
11     final int sira;
12     String kullanici = izYaz("[O-O1] ornek alan baslaticisi (kullanici)");
13     { izYaz("[O-O2] ornek baslatma blogu"); }
14
15     Oturum(String kullanici) {
16         izYaz("[O-Y1] yapici govdesi");
17         acilanToplam++;
18         this.sira = acilanToplam;
19         this.kullanici = kullanici;
20     }
```

Kod 6.8 — Gözlenen sınıf: her aşamaya bir iz bırakılmış

Sınıfın **3–8. satırları** sınıf düzeyini, **10–20. satırları** ise nesne düzeyini oluşturur. Şimdi programı çalıştırıp hangi izin ne zaman düştüğüne bakalım:

```
▸ TERMINAL — java StatikBlokOrnegi
=== 1) main basladi. Hicbir sinif YUKLENMEDI ===

=== 2) Yapilandirma sinifina ILK erisim ===
      [Y-S1] Yapilandirma static blogu calisti
  Yapilandirma.surum() = 2026.2-guz
=== 3) Ikinci erisim: static blok ARTIK calismaz ===
  Yapilandirma.surum() = 2026.2-guz
  okumaSayisi = 2

=== 4) Ilk new Oturum: once SINIF, sonra NESNE ===
      [O-S1] Oturum static blogu calisti (AZAMI_ESZAMANLI=3)
      [O-O1] ornek alan baslaticisi (kullanici)
      [O-O2] ornek baslatma blogu
      [O-Y1] yapici govdesi
  uretildi -> Oturum#1 (elif)

=== 5) Ikinci new: static bolum YOK, ornek bolum VAR ===
      [O-O1] ornek alan baslaticisi (kullanici)
      [O-O2] ornek baslatma blogu
      [O-Y1] yapici govdesi
  uretildi -> Oturum#2 (mert)
```

Çıktıdan çıkarılacak dört sonuç vardır:

1. **`main` başladığında hiçbir sınıf yüklenmemişti.** İlk satır, herhangi bir `static` blok çıktısından öncedir.
2. **`Yapilandirma` sınıfı ilk `surum()` çağrısında yüklendi.** İkinci çağrıda `[Y-S1]` satırı hiç görünmez; sınıf zaten yüklüdür.
3. **`Oturum` sınıfının `static` bloğu yalnızca ilk `new` çağrısında çalıştı.** İkinci `new`'de `[O-S1]` yoktur, ama `[O-O1]`, `[O-O2]` ve `[O-Y1]` yeniden çalışmıştır.
4. **Sıra her zaman aynıdır:** önce sınıf düzeyi (bir kez), sonra nesne düzeyi (her `new` için). Hafta 5'te öğrendiğimiz nesne başlatma sırası, bu daha büyük sıranın ikinci yarısıdır.

Sıra	Ne çalışır?	Ne zaman?	Kaç kez?
0	Sınıf yüklenir	İlk gerçek kullanımda	Sınıf başına 1
1	`static` alan başlatıcıları	Sınıf yüklenirken	Sınıf başına 1
2	`static` bloklar	Sınıf yüklenirken	Sınıf başına 1
3	Bellek ayrılır, alanlar sıfırlanır	Her `new`	Nesne başına 1
4	Örnek alan başlatıcıları	Her `new`	Nesne başına 1
5	Örnek başlatma blokları	Her `new`	Nesne başına 1
6	Yapıcı gövdesi	Her `new`	Nesne başına 1

Tablo 6.5 — Sınıf düzeyi ve nesne düzeyi başlatmanın tam sırası

▲ DİKKAT

1 ve 2 numaralı adımlar kaynak koddaki yazım sırasını izler. Bu, sezgiye aykırı bir tuzak üretir: bir `static` alanı, kendisinden **sonra** bildirilen bir alana dayanarak başlatamazsınız. `Ayarlar` sınıfındaki sayaç bildirimi tam da bu nedenle `ORNEK` bildiriminin üstündedir; sıra ters olsaydı yapıcının artırdığı sayaç `= 0` başlatıcısı sıfırlardı.

4.2. `static` Bloğun Asıl İşi: Kurulum Doğrulaması

`static` blokların en değerli kullanımı, tek satırla kurulamayacak bir sabiti hazırlamak ve **kurulumun tutarlılığını doğrulamaktır**. Laboratuvar `SinavKurallari` sınıfı bunu yapar:

SinavKurallari.java

```
1      // static blok : sinif yuklenirken kurulumun tutarliligini dogrular
2      static {
3          double toplam = VIZE_AGIRLIGI + ODEV_AGIRLIGI + LAB_AGIRLIGI + FINAL_AGIRLIGI;
4          if (Math.abs(toplam - 1.0) > 1e-9) {
5              throw new ExceptionInInitializerError(
6                  "Agirliklar toplami 1.0 degil: " + toplam);
7          }
8          if (ESIKLER.length + 1 != HARFLER.length) {
9              throw new ExceptionInInitializerError("Esik/harf sayisi uyusmuyor.");
10         }
11         System.out.println("  [static blok] " + DERS_KODU
12                            + " kurallari dogrulandi (agirlik toplami=" + toplam + ")");
13     }
```

Kod 6.9 — `static` blok: ağırlıklar toplamı 1.0 değilse sınıf hiç yüklenmez

3–7. satırlar bir güvenlik kilidi kurar. Bir gün biri `VIZE_AGIRLIGI` değerini `0.35` yapıp diğerlerini güncellemeyi unutursa, sınıf ilk kullanıldığı anda `ExceptionInInitializerError` ile programı durdurur. Alternatif, dönem boyunca sessizce yanlış not hesaplamaktır.

```
▸ TERMINAL — java Gorev3_KurallarDemo (sınıf yükleme anı)
=== 1) Derleme zamani sabiti sinifi YUKLEMEZ ===
  DERS_KODU = YZM205
  static blok CIKTISI HENUZ YOK: DERS_KODU bir
  static final String'tir ve degeri derleme aninda
  cagiran sinifin icine GOMULMUSTUR.

=== 2) Ilk METOT cagrisi: sinif SIMDI yuklenir ===
  SinavKurallari.esikler() cagriliyor...
  [static blok] YZM205 kurallari dogrulandi (agirlik toplami=1.0)
```

Çıktının **ilk beş satırı** ince ama önemli bir ayrıntıyı ortaya koyar: `DERS_KODU` sabitine erişmek sınıfı **yüklemedi**. Bunun nedeni, `static final` bir `String` ya da ilkel sabitin değerinin derleme anında çağıran sınıfın içine gömülmesidir. Sınıf ancak gerçek bir metot çağrıldığında (`esikler()`) yüklenmiştir.

ⓘ BİLGİ

Buna *derleme zamanı sabiti* (compile-time constant) denir ve pratik bir sonucu vardır: bir kütüphanedeki `public static final` sabitin değerini değiştirirseniz, o sabiti kullanan kodun **yeniden derlenmesi** gerekir.

> Yalnızca kütüphaneyi güncellemek yetmez; eski değer çağıran sınıfın içinde gömülü kalır.

4.3. Yardımcı (Utility) Sınıflar

Bazı sınıfların hiç durumu yoktur. `Math` sınıfını düşününüz: `Math.sqrt(144.0)` çağrısı için bir `Math` nesnesi üretmek anlamsız olurdu, çünkü iki `Math` nesnesi arasında hiçbir fark olamazdı. Böyle sınıflara **yardımcı sınıf** denir ve üç kurala uyarlar.



Şekil 6.5 — Yardımcı sınıfın üç kuralı ve Java kitaplığındaki örnekleri

MatematikYardimcisi.java

```java
public final class MatematikYardimcisi {

    // ---- static final sabitler ----
    public static final double PI_YAKLASIK = 3.14159265358979;
    public static final double ALTIN_ORAN  = 1.61803398874989;

    // static blok: karmasik sabit kurulumu icin
    private static final int[] FAKTORIYEL_ONBELLEK;
    static {
        FAKTORIYEL_ONBELLEK = new int[13];   // 12! int'e sigar, 13! sigmaz
        FAKTORIYEL_ONBELLEK[0] = 1;
        for (int i = 1; i < FAKTORIYEL_ONBELLEK.length; i++) {
            FAKTORIYEL_ONBELLEK[i] = FAKTORIYEL_ONBELLEK[i - 1] * i;
        }
        System.out.println("  [static blok] faktoriyel onbellegi kuruldu "
                           + "(0!..12!)");
    }

    /* Yapiciyi private yapmak, "bu sinifin nesnesi anlamsizdir"
     * bilgisini DERLEYICIYE soyler. new MatematikYardimcisi() yazan
     * kod su hatayi alir:
     *   The constructor MatematikYardimcisi() is not visible
     */
    private MatematikYardimcisi() {
        throw new AssertionError("Yardimci sinif ornegi uretilemez.");
    }
```

Kod 6.10 — Üç kuralın uygulanışı: `final` sınıf, `static` üyeler, `private` yapıcı

1. satır sınıfı `final` yapar: miras alınamaz. **9–17. satırlar** bir `static` bloğun tipik kullanımıdır — 13 elemanlı faktöriyel önbelleği tek bir atamayla kurulamazdı. **24–26. satırlardaki** `private` yapıcı ise kapıyı kapatır; `new MatematikYardimcisi()` yazan kod `The constructor MatematikYardimcisi() is not visible` hatasını alır.

```
▸ TERMINAL — java YardimciDemo (tembel yükleme ve static metotlar)
=== 1) main basladi. MatematikYardimcisi henuz KULLANILMADI ===
  Bu satir static blogun CIKTISINDAN ONCE gorunuyor.

=== 2) Ilk erisim: sinif simdi yuklenir ===
  MatematikYardimcisi.kare(7) cagriliyor...
  [static blok] faktoriyel onbellegi kuruldu (0!..12!)
  sonuc = 49

=== 3) Ikinci erisim: static blok ARTIK calismaz ===
  faktoriyel(6)  = 720
  faktoriyel(12) = 479001600

=== 4) Nesne URETMEDEN calisan metotlar ===
  dizi        = [68, 91, 74, 55, 88, 79]
  enBuyuk     = 91
  ortalama    = 75.83
  obeb(84,36) = 12
  2..30 asallari: 2 3 5 7 11 13 17 19 23 29
```

İlk iki satır tembel yüklemeyi bir kez daha doğruluyor: `main` başladıktan **sonra**, `kare(7)` çağrısıyla birlikte `static` blok çalıştı. Sonraki çağrılarda ise çalışmadı.

Kural	Nasıl yazılır?	Uygulanmazsa ne olur?
Tüm üyeler `static`	`public static double kmToMil(double km)`	Sınıfın nesnesi anlamlı hâle gelir; tasarım niyeti bulanıklaşır
Yapıcı `private`	`private BirimCevirici() { ... }`	Derleyici parametresiz yapıcı ekler; anlamsız nesneler üretilebilir
Sınıf `final`	`public final class BirimCevirici`	Sınıf miras alınabilir görünür; oysa alt sınıf üst yapıcıyı çağıramaz
Yapıcı gövdesinde `throw`	`throw new AssertionError(...)`	Sınıfın kendi içinden ya da yansımayla (reflection) nesne üretilebilir

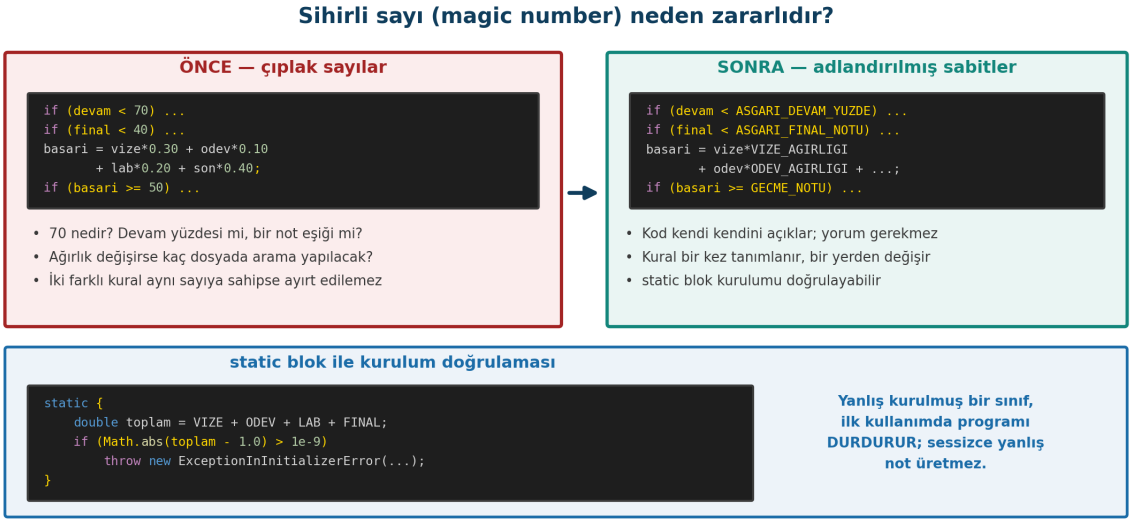
Tablo 6.6 — Yardımcı sınıfın kuralları ve uygulanmama bedelleri

▲ DİKKAT

Yardımcı sınıf **kolay bir kaçış yolu değildir**. Nesne üretmekten kurtulmak için her şeyi `static` yapmak, nesne yönelimli tasarımın tamamını devre dışı bırakır ve Hafta 7–10'da göreceğimiz kalıtım ve çok biçimlilik araçlarını kullanılamaz hâle getirir. Sınama sorusu şudur: *"Bu sınıftan iki nesne üretsem aralarında bir fark olur muydu?"* Yanıt evetse yardımcı sınıf yanlış tercihtir.

4.4. `static final` Sabitler ve Sihirli Sayılar

Kodun içine doğrudan yazılmış çıplak sayılara **sihirli sayı** (magic number) denir. Yazıldığı gün anlamı bellidir; altı ay sonra değildir.



Şekil 6.6 — Sihirli sayıdan adlandırılmış sabite ve kurulum doğrulamasına

Sabitler.java

```java
// ---- Derleme zamani sabitleri : ilkel + String, deger yerine gomulur ----
public static final int    AZAMI_DERS_SAYISI = 8;
public static final int    ASGARI_DEVAM_YUZDE = 70;
public static final double VIZE_AGIRLIGI  = 0.40;
public static final double FINAL_AGIRLIGI = 0.60;
public static final String KURUM = "KTU Of Teknoloji Fakultesi";

// ---- Calisma zamani sabiti : referans, adres degismez ICERIK degisebilir ----
public static final int[] GECME_ESIKLERI = { 50, 60, 70, 80, 90 };
```

Kod 6.11 — Adlandırılmış sabitler: iki farklı türden

```
▸ TERMINAL — java SabitlerDemo (sihirli sayı ve sabit)
=== 1) Sihirli sayi ile adlandirilmis sabit ===
  sihirliHesap(65, 78)        = 72.8
  Sabitler.basariNotu(65, 78) = 72.8
  Ayni sonuc; fark OKUNABILIRLIKTE ve TEK NOKTADAN
  degistirilebilmesindedir.

=== 2) Sabitlerin degeri ===
  KURUM              = KTU Of Teknoloji Fakultesi
  AZAMI_DERS_SAYISI  = 8
  ASGARI_DEVAM_YUZDE = 70
  VIZE + FINAL       = 1.0
```

İki hesap da aynı sonucu üretti. Fark hesapta değil, **kodun okunabilirliğinde ve değiştirilebilirliğindedir**. Ağırlıklar değişirse sihirli sürümde her kullanım yeri tek tek bulunmalıdır; sabitli sürümde tek satır yeterlidir.

Ölçüt	Sihirli sayı	`static final` sabit
Anlamın kaynağı	Okuyucunun tahmini	Sabitin adı
Değişiklik maliyeti	Tüm kullanım yerleri	Tek satır
Aynı değerli iki farklı kural	Ayırt edilemez	İki ayrı ad
Yazım hatası riski	`0.30` yerine `0.03`	Derleyici adı denetler
Kurulum doğrulaması	Mümkün değil	`static` blokta yapılabilir
Belgelendirme	Yorum satırı gerekir	Ad zaten belgedir

Tablo 6.7 — Sihirli sayı ile adlandırılmış sabitin karşılaştırması

4.5. `final`'ın Sınırı: Referansı Korur, İçeriği Korumaz

Hafta 5'te dizilerde `clone()` kullanmayı öğrenirken bu konuya dokunmuştuk. `static final` bağlamında sonucu daha da tehlikelidir: `final` yalnızca alanın **hangi nesneyi gösterdiğini** dondurur; gösterdiği nesnenin içeriğini korumaz.



Şekil 6.7 — `final` bir diziyi gerçekten korumaz — savunmalı kopya gerekir

```
▸ TERMINAL — java SabitlerDemo (bozulan sabit)
=== 4) final REFERANSI korur, ICERIGI korumaz ===
  esikler once : 50 60 70 80 90
  Sabitler.GECME_ESIKLERI[0] = 45;  // DERLENIR!
  esikler sonra: 45 60 70 80 90
  harfNotu(47) = DD   <- sabit disaridan BOZULDU
  Cozum: diziyi private tutup kopya donduren bir
  static metot yazmaktir (List.of ya da clone).
```

Çıktı, `public static final int[]` bildiriminin verdiği güvence duygusunun yanıltıcı olduğunu gösterir: sınıfın dışındaki bir kod, geçme eşiğini `50`'den `45`'e indirebildi ve `harfNotu(47)` artık `DD` döndürdü. Derleyici hiçbir uyarı vermedi.

Çözüm, diziyi `private` tutmak ve dışarıya **kopya** vermektir:

```
● SinavKurallari.java
```

```
    // Disariya kopya verilir; asil dizi private kalir
    private static final int[]    ESIKLER = { 50, 60, 70, 80, 90 };
    private static final String[] HARFLER = { "FF", "DD", "CC", "BB", "BA", "AA" };
```

Kod 6.12 — Asıl dizi `private`

```
SinavKurallari.java
    /** Savunmali kopya: cagiran kod diziyi bozarsa sabit etkilenmez. */
    public static int[] esikler() {
        return ESIKLER.clone();
    }
```

Kod 6.13 — Savunmalı kopya: dışarı çıkan her zaman kopyadır

```
▸ TERMINAL — java Gorev3_KurallarDemo (savunmalı kopya)

=== 5) Savunmali kopya: sabit disaridan bozulamaz ===
  esikler()  : 50 60 70 80 90
  kopya[0] = 5 yapildi.
  esikler()  : 50 60 70 80 90     <- ASIL dizi bozulmadi
```

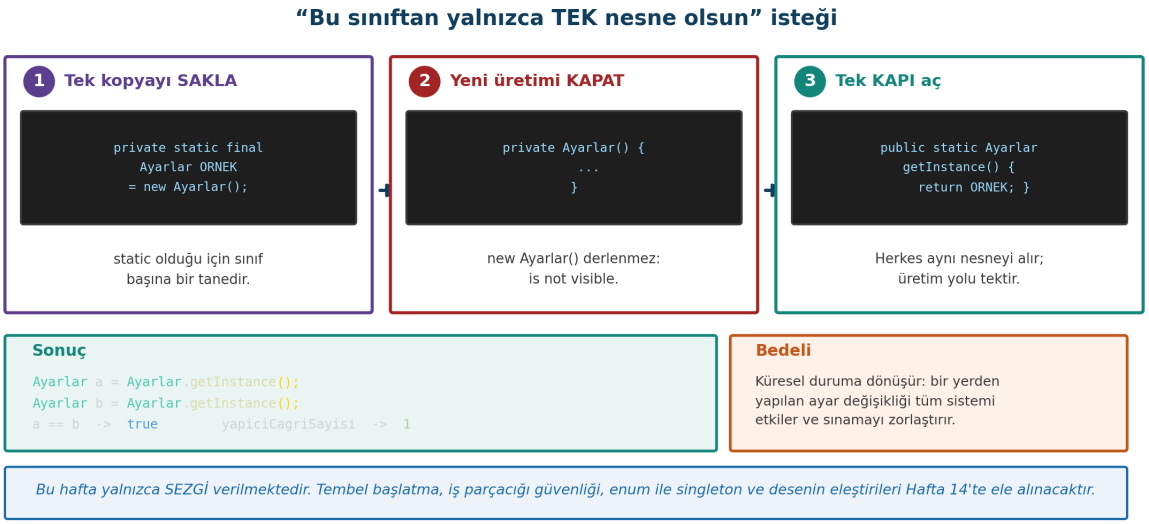
Aynı saldırı bu kez sonuçsuz kaldı: `kopya[0] = 5;` ataması yalnızca kopyayı değiştirdi, asıl dizi bozulmadı.

Sabitin türü	`public static final` güvenli mi?	Gerekçe
`int`, `double`, `boolean` gibi ilkeller	**Evet**	Değer alanın içindedir; `final` onu dondurur
`String`	**Evet**	`String` değişmezdir (immutable)
`LocalDate`, `Integer` gibi değişmez sınıflar	**Evet**	İçerikleri değiştirilemez
Dizi (`int[]`, `String[]`)	**Hayır**	Hücreler yazılabilir; `private` + kopya gerekir
Koleksiyon (`ArrayList`, `HashMap`)	**Hayır**	Eleman eklenip çıkarılabilir (Hafta 13)
Kendi yazdığınız değişebilir sınıf	**Hayır**	Setter'ları çağrılabilir

Tablo 6.8 — Hangi türler için `public static final` güvenlidir?

4.6. Tekil (Singleton) Sezgisi

Bu haftanın araçları bir araya geldiğinde ilginç bir kapı açılır: bir sınıftan **yalnızca tek bir nesne** üretilmesini zorunlu kılabiliriz. Bunun için Hafta 5'te öğrendiğimiz `private` yapıcıyla bu haftanın `static` alanını birleştirmek yeterlidir.



Şekil 6.8 — Tekil desenin üç parçası: sakla, kapat, tek kapı aç

Ayarlar.java

```
1  public class Ayarlar {
2
3      /* DIKKAT - BILDIRIM SIRASI ONEMLIDIR.
4       * static alan baslaticilari YUKARIDAN ASAGIYA calisir. Sayac
5       * bildirimi ORNEK bildiriminin ALTINDA olsaydi, yapici sayaci
6       * 1 yapar, hemen ardindan "= 0" baslaticisi onu SIFIRLARDI.
7       * Bu, Hafta 5'te ogrenilen baslatma sirasinin static karsiligidir. */
8      private static int yapiciCagriSayisi = 0;
9
10     // 1) Tek nesne, sinif yuklenirken bir kez uretilir (erken baslatma)
11     private static final Ayarlar ORNEK = new Ayarlar();
12
13     // Nesne duzeyi durum: ayarlarin kendisi
14     private String tema;
15     private int    otomatikKayitDakika;
16
17     // 2) private yapici: disaridan new Ayarlar() YAZILAMAZ
18     private Ayarlar() {
19         yapiciCagriSayisi++;
20         this.tema = "acik";
21         this.otomatikKayitDakika = 5;
22         System.out.println("  [yapici] Ayarlar nesnesi uretildi "
23                            + "(cagri sayisi=" + yapiciCagriSayisi + ")");
24     }
25
26     // 3) Tek kapi
27     public static Ayarlar getInstance() {
28         return ORNEK;
29     }
```

Kod 6.14 — Üç parçanın bir arada uygulanışı

8. satırdaki sayaç bildiriminin **11. satırdaki** `ORNEK` bildiriminden önce gelmesi zorunludur; nedenini Bölüm 4.1'de görmüştük. **18. satırdaki** `private` yapıcı dışarıdan üretimi kapatır, **27–28. satırlardaki** `getInstance()` ise tek kapıyı açar.

```
▸ TERMINAL — java AyarlarDemo (tek nesne güvencesi)
=== 1) main basladi ===
  Ayarlar sinifi henuz KULLANILMADI; yapici ciktisi YOK.

=== 2) Ilk getInstance() : sinif SIMDI yuklenir ===
  [yapici] Ayarlar nesnesi uretildi (cagri sayisi=1)
  a = Ayarlar[tema=acik, otomatikKayit=5 dk]

=== 3) Ikinci ve ucuncu cagri ===
  yapiciCagriSayisi = 1
  a == b ? true
  b == c ? true
  Uc cagri yapildi ama yapici YALNIZCA BIR KEZ calisti.

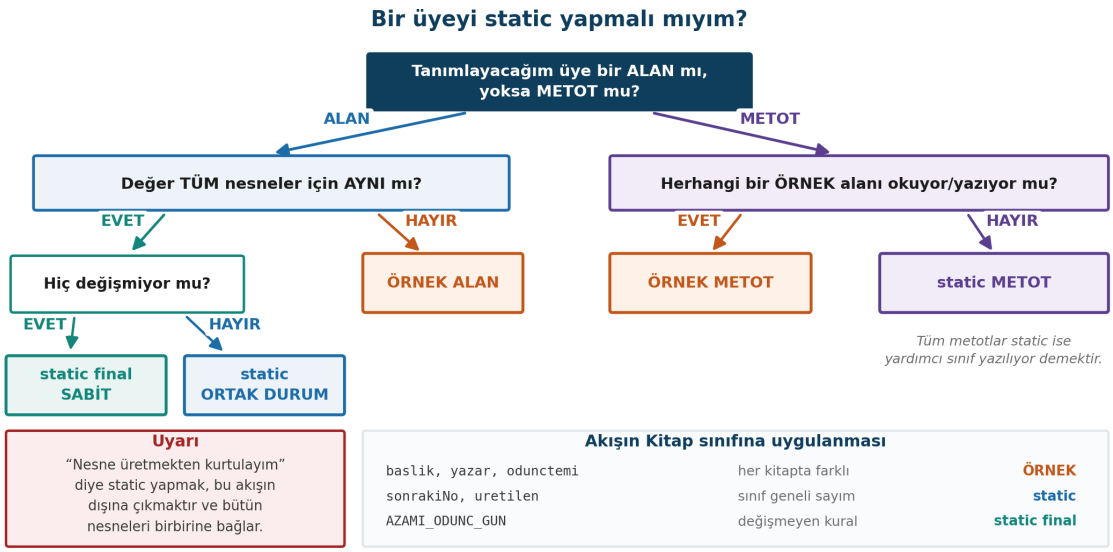
=== 4) Birinden yapilan degisiklik hepsinde gorunur ===
  a -> Ayarlar[tema=koyu, otomatikKayit=15 dk]
  b -> Ayarlar[tema=koyu, otomatikKayit=15 dk]
  c -> Ayarlar[tema=koyu, otomatikKayit=15 dk]
```

Üç kez `getInstance()` çağrıldı, ama yapıcı yalnızca bir kez çalıştı ve üç değişken de aynı nesneyi gösterdi. Bu yüzden `a` üzerinden yapılan tema değişikliği `b` ve `c` üzerinden de görülüyor.

> **▲ DİKKAT**
>
> Bu desen bir çözüm olduğu kadar bir **risktir**. Tek nesne, programın her yerinden erişilebilen küresel bir duruma dönüşür; bir yerden yapılan ayar değişikliği tüm sistemi etkiler ve birim sınamalarını zorlaştırır. Tembel başlatma, iş parçacığı güvenliği, `enum` ile singleton ve desenin eleştirileri Hafta 14'te ele alınacaktır.

4.7. `static` mi, Örnek Üyesi mi? Karar Akışı

Bu bölümdeki tüm bilgiler tek bir karar akışında toplanabilir.



Şekil 6.9 — Bir üyeyi `static` yapmalı mıyım?

Akışın çıkışları birbirini dışlamaz; bir sınıf aynı anda hem `static final` sabitlere, hem `static` sayaçlara, hem de örnek alanlara sahip olabilir. `Kitap` sınıfı tam olarak böyledir.

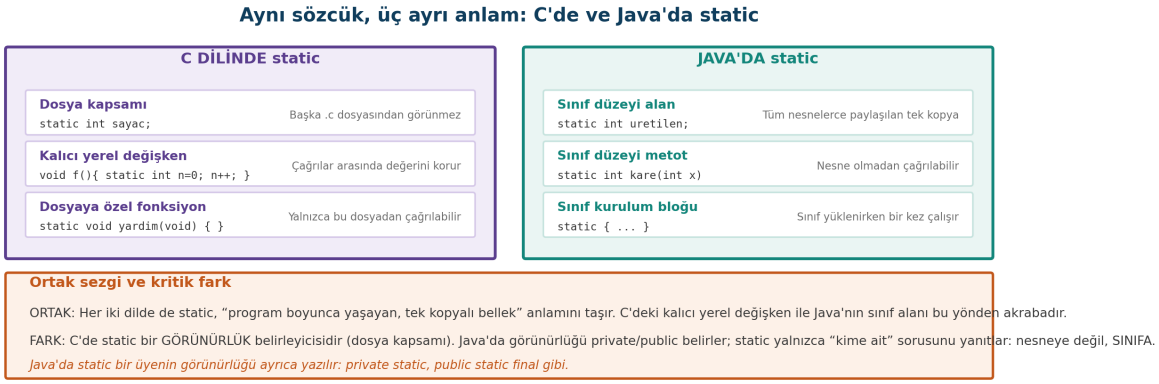
Soru	Yanıt evetse	Örnek
Değer tüm nesneler için aynı ve hiç değişmiyor mu?	`static final`	`AZAMI_ODUNC_GUN`, `Math.PI`
Değer tüm nesneler için aynı ama değişiyor mu?	`static`	`sonrakiNo`, `uretilen`
Değer her nesnede farklı mı?	Örnek alan	`baslik`, `odunctemi`
Metot herhangi bir örnek alanı okuyor/yazıyor mu?	Örnek metodu	`oduncVer()`, `getBaslik()`
Metot yalnızca parametrelerden sonuç üretiyor mu?	`static` metot	`numaraUret()`, `Math.max()`
Sınıfın **hiçbir** örnek üyesi yok mu?	Yardımcı sınıf	`BirimCevirici`, `Arrays`

Tablo 6.9 — Karar akışının soru–yanıt biçiminde özeti

5. C'den Java'ya Köprü

⇄ C DİLİNDEN JAVA'YA KÖPRÜ

C dilinde `static` anahtar sözcüğü zaten vardır ve iki ayrı iş yapar: bir değişkeni ya da fonksiyonu dosya kapsamına hapsetmek (**görünürlük**), ve bir yerel değişkeni çağrılar arasında yaşatmak (**ömür**). Java'da ise `static` görünürlükle hiç ilgilenmez — onu `private`/`public` belirler. Java'da `static` yalnızca tek bir soruyu yanıtlar: *bu üye kime ait — nesneye mi, sınıfa mı?*



Şekil 6.10 — Aynı sözcük, farklı sorumluluklar: C'de ve Java'da `static`

kitap.c

```c
/* C - static'in iki ayri isi */
static int toplam_kitap = 0;          /* (1) dosya kapsami: baska .c goremez */

static void yardimci(void) { }        /* (1) dosyaya ozel fonksiyon */

int sonraki_numara(void) {
    static int sayac = 0;             /* (2) OMUR: cagrilar arasinda yasar */
    sayac++;                          /*     ama yalnizca bu fonksiyondan gorunur */
    return sayac;
}

int main(void) {
    printf("%d\n", sonraki_numara());   /* 1 */
    printf("%d\n", sonraki_numara());   /* 2 */
    return 0;
}
```

Kod 6.15 — C'de `static`: görünürlük ve ömür

C'deki ikinci kullanım (kalıcı yerel değişken), Java'daki `static` alana en yakın akrabadır: her ikisi de program boyunca yaşayan, tek kopyalı bir bellek hücresidir. Ancak C'de bu hücre bir fonksiyona gizlenmiştir ve dışarıdan okunamaz; Java'da ise sınıfın bir üyesidir ve erişimi ayrıca belirlenir.

```java
// Java - ayni is, ama sorumluluklar AYRI yazilir
public class Kitap {
    private static int sonrakiNo = 1;      // private: gorunurluk
```

```
4                                           // static : sinifa ait
5       private static String numaraUret() {    // hem private hem static
6           return String.format("KTP-%05d", sonrakiNo++);
7       }
8
9       public static int uretilenSayisi() { return uretilen; }  // disariya acik
10  }
```

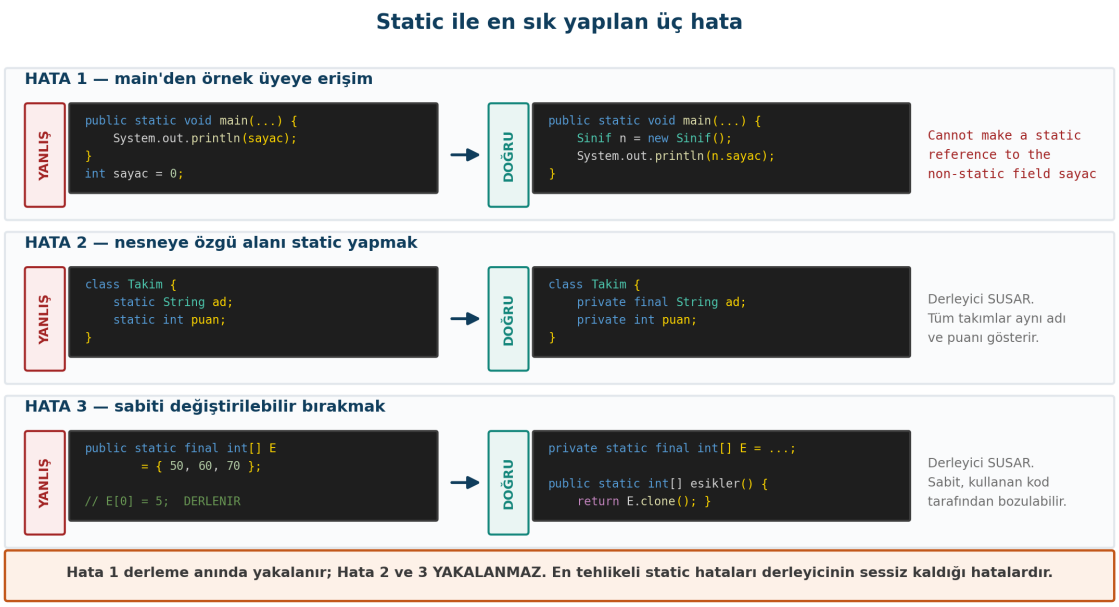
Kod 6.16 — Java'da görünürlük ve aidiyet ayrı anahtar sözcüklerle yazılır

Konu	C dilinde	Java'da
`static` neyi belirler?	Görünürlük **ve** ömür	Yalnızca aidiyet (nesne / sınıf)
Görünürlüğü ne belirler?	`static` ve başlık dosyası	`private` / `public` / `protected`
Genel (global) değişken	Dosya düzeyinde bildirilir	Yoktur — her şey bir sınıfın üyesidir
Kalıcı yerel değişken	`static int n = 0;` fonksiyon içinde	Yoktur — `static` alan kullanılır
Kurulum kodu	`main` başında elle çağrılır	`static { ... }` bloğu
Kurulum ne zaman çalışır?	Programcının yazdığı yerde	Sınıf ilk kullanıldığında (tembel)
Sabit tanımı	`#define` ya da `const int`	`static final` (tür denetimli)
Ad çakışması	Bağlayıcı (linker) düzeyinde olasıdır	Sınıf adı önek olduğu için oluşmaz

Tablo 6.10 — C'deki `static` ile Java'daki `static`'in karşılaştırması

> **ⓘ BİLGİ**
>
> `#define AZAMI_KREDI 45` ile `public static final int AZAMI_KREDI = 45;` arasındaki fark yalnızca sözdizimi değildir. `#define` bir **metin değiştirmedir**; önişlemci `AZAMI_KREDI` yazan her yere `45` yazar ve tür denetimi yapılmaz. Java sabiti ise türlüdür, hata ayıklayıcıda adıyla görünür ve yanlış türde kullanılırsa derleyici uyarır.

6. Sık Yapılan Hatalar



Şekil 6.11 — `static` ile en sık yapılan üç hata ve düzeltilmiş hâlleri

6.1. `main` İçinden Örnek Üyeye Erişmek

✕ HATA 1 — `static` bağlamdan örnek üyeye erişim

`main` metodu `static`'tir; bu yüzden içinden doğrudan bir örnek alanına ya da örnek metoduna erişemez. Derleyici `Cannot make a static reference to the non-static field ...` iletisini verir. Çözüm, alanı gelişigüzel `static` yapmak **değil**, `main` içinde bir nesne üretip ona erişmektir.

```java
// YANLIS
public class Sayac {
    int deger = 0;                      // ornek alan
    public static void main(String[] a) {
        deger++;                        // DERLEME HATASI
    }
}

// DOGRU
public class Sayac {
    int deger = 0;
    public static void main(String[] a) {
        Sayac s = new Sayac();          // once nesne
        s.deger++;                      // sonra erisim
    }
}
```

Kod 6.17 — Yanlış ve doğru kullanım yan yana

6.2. Nesneye Özgü Alanı `static` Yapmak

> ✖ **HATA 2 — derleyicinin sessiz kaldığı hata**
>
> Derleme hatasından kurtulmak için alanın önüne `static` yazmak, bu haftanın **en tehlikeli** hatasıdır. Program derlenir, çalışır ve yanlış sonuç üretir. Nesneye özgü bir alan `static` yapıldığında tüm nesneler tek bir değeri paylaşmaya başlar.

Ders örneğindeki `KotuSepet` sınıfı bu hatayı bilerek yapar:

StatikCozumleme.java

```
1       // ---- yanlis tasarim ornegi : static sayac ile ortak durum ----
2       static class KotuSepet {
3           // HATA: sepet toplami STATIC yapilmis -> tum sepetler ayni toplami paylasir
4           static double toplam = 0.0;
5           String sahibi;
6
7           KotuSepet(String sahibi) { this.sahibi = sahibi; }
8
9           void ekle(double fiyat) { toplam += fiyat; }
10
11          @Override
12          public String toString() { return sahibi + " sepeti toplam=" + toplam; }
13      }
```

Kod 6.18 — Sepet toplamı yanlışlıkla `static` yapılmış

```
▸ TERMINAL — java StatikCozumleme (bozuk ve düzeltilmiş sürüm)
=== 1) Static durum yanlis kullanilirsa ===
  Elif sepeti toplam=165.0
  Mert sepeti toplam=165.0
  Iki sepet de 165.0 gosteriyor: TEK toplam var.

=== 2) Duzeltilmis surum ===
  Elif sepeti toplam=120.0
  Mert sepeti toplam=45.0
  IyiSepet.acilanSepet = 2   (sinif geneli olcum: static DOGRU kullanim)
```

Elif 120 TL'lik, Mert 45 TL'lik alışveriş yaptı; ama iki sepet de 165 TL gösteriyor. Düzeltilmiş `IyiSepet` sınıfında `toplam` örnek alanıdır, `acilanSepet` ise `static` kalmıştır — çünkü “kaç sepet açıldı” gerçekten sınıf geneli bir ölçümdür.

6.3. Sabiti Değiştirilebilir Bırakmak

> ✖ **HATA 3 — sabit görünümlü değişken**
>
> `public static final int[] ESIKLER = {...};` bildirimi bir sabit **gibi görünür**, ama değildir. Dizinin hücreleri sınıfın dışından yazılabilir ve derleyici uyarmaz. Dizi ve koleksiyon sabitleri `private` tutulup kopya döndürülmelidir (Bölüm 4.5).

6.4. Her Şeyi `static` Yaparak Nesnelerden Kaçmak

✕ HATA 4 — nesne yöneliminden kaçış

C'den gelen öğrencilerin sık düştüğü tuzak, tüm alanları ve metotları `static` yapıp programı bir fonksiyon yığını gibi yazmaktır. Bu, Java'da derlenir ve çalışır; ancak nesne yönelimli tasarımın tamamını devre dışı bırakır. Hafta 7'den itibaren göreceğimiz kalıtım ve çok biçimlilik, `static` üyelerle kullanılamaz.

6.5. `static` Alanı Nesne Üzerinden Çağırmak

✕ HATA 5 — yanıltıcı çağırma biçimi

`k3.uretilenSayisi()` yazımı derlenir ve doğru sonucu verir; ama okuyucuya "bu, `k3` nesnesinin üretilen sayısı" gibi yanlış bir izlenim verir. `Kitap.uretilenSayisi()` yazımı tercih edilmelidir. Aynı gerekçeyle `null` bir referans üzerinden `static` çağrı da hata vermez — çünkü nesneye hiç bakılmaz (Bölüm 3.4).

6.6. `static` Bloğu Sıraya Dikkat Etmeden Yazmak

✕ HATA 6 — bildirim sırasını önemsememek

`static` alan başlatıcıları ve `static` bloklar kaynak koddaki yazım sırasını izler. Bir `static` alanı, kendisinden **sonra** bildirilen bir alana dayanarak başlatmak derleme hatası ya da sessiz bir `0`/`null` üretir. Bildirimlerin sırası kodun davranışının parçasıdır.

6.7. Derleyici İletileri Sözlüğü

Derleyici iletisi	Ne demek?	Ne yapmalı?
`Cannot make a static reference to the non-static field x`	`static` bir bağlamdan örnek alanına erişiliyor	Nesne üretin ya da alanın gerçekten `static` olması gerektiğine karar verin
`Cannot make a static reference to the non-static method m()`	`static` bir bağlamdan örnek metoduna erişiliyor	Aynı çözüm; metodun `static` olup olmayacağını Bölüm 4.7 akışıyla belirleyin
`Cannot use this in a static context`	`static` metotta `this` yazılmış	`this`'i kaldırın; `static`'in `this`'i yoktur
`The final field X cannot be assigned`	`final` alana ikinci atama yapılıyor	Atamayı yalnızca bildirimde ya da tek yerde yapın
`The blank final field X may not have been initialized`	`static final` alan `static` blokta atanmamış	Tüm yollarda atandığından emin olun
`The constructor X() is not visible`	Yardımcı sınıfın `private` yapıcısı çağrılıyor	Nesne üretmeyin; `static` metotları doğrudan çağırın

Derleyici iletisi	Ne demek?	Ne yapmalı?
`ExceptionInInitializerError`	`static` blok istisna fırlattı	Kurulum doğrulamanızın gerekçesini okuyun; sabitleri düzeltin
`NoClassDefFoundError` (ikinci kullanımda)	Sınıf yüklenirken hata almış, yükleme yarım kalmış	Asıl nedeni ilk `ExceptionInInitializerError` iletisinde arayın

Tablo 6.11 — Bu haftaya özgü derleme ve çalışma zamanı hataları

7. İyi Tasarım İlkeleri

Bu bölümdeki maddeler, dönem boyunca yazacağınız her sınıfta uygulamanız beklenen kod yazım standartlarıdır.

1. **Bir üyeyi `static` yapmadan önce gerekçesini yazın.** Gerekçe "derleme hatasından kurtulmak" ise `static` yanlış çözümdür.
2. **Alanları üç grup hâlinde bildirin:** önce `static final` sabitler, sonra `static` alanlar, en sonra örnek alanları. Her grubun başına bir yorum satırı koyun.
3. **`static` alanları `private` tutun.** Sayaç ve durum alanları dışarıya `static` bir getter ile açılmalıdır; doğrudan yazılabilir olmamalıdır.
4. **Sabitleri `public static final` ve SCREAMING_SNAKE_CASE ile yazın.** Yalnızca ilkel türler, `String` ve değişmez nesneler için `public` yapın.
5. **Dizi ve koleksiyon sabitlerini asla `public` yapmayın.** `private` tutup `clone()` ya da salt-okunur bir sarmalayıcı döndürün.
6. **`static` üyelere sınıf adı üzerinden erişin.** `nesne.staticUye` yazımı yanıltıcıdır ve IDE uyarısı üretir.
7. **`static` bloğu yalnızca gerektiğinde kullanın.** Tek satırlık bir atama yeterliyse alan başlatıcısı tercih edilir.
8. **`static` bloğu kurulum doğrulaması için kullanın.** Tutarsız bir kurulum programı erken durdurmalıdır; sessizce yanlış çalışmamalıdır.
9. **`static` alan bildirimlerinin sırasına dikkat edin.** Birbirine dayanan başlatıcılarda sıra kodun davranışının parçasıdır.
10. **Yardımcı sınıfları üç kurala uydurun:** tüm üyeler `static`, yapıcı `private`, sınıf `final`.
11. **Nesneye özgü hiçbir veriyi `static` yapmayın.** Ad, bakiye, kadro, durum gibi alanlar nesneye aittir.
12. **Değiştirilebilir `static` durumu en aza indirin.** Küresel durum, sınamayı zorlaştırır ve çok iş parçacıklı ortamda hataya açıktır.

7.1. Adlandırma ve Yazım Kuralları

Öğe	Kural	Örnek
`static final` sabit	SCREAMING_SNAKE_CASE	`AZAMI_ODUNC_GUN`
`static` sayaç	camelCase, çoğunlukla `sonraki`/`toplam` öneki	`sonrakiNo`, `toplamKredi`
`static` getter	Alan adıyla aynı, parantezli	`uretilenSayisi()`
Yardımcı sınıf adı	İşi anlatan ad; `Yardimcisi`, `Cevirici`, `Araclari` sonekleri yaygın	`MatematikYardimcisi`, `BirimCevirici`
Anahtar sözcük sırası	erişim → `static` → `final` → tür → ad	`public static final int X`
Bildirim sırası	sabitler → `static` alanlar → örnek alanları → yapıcılar → metotlar	Bkz. `Kitap.java`
`static` blok konumu	İlgili alanların hemen ardına	Bkz. `SinavKurallari.java`

Tablo 6.12 — `static` bağlamında Java adlandırma gelenekleri

8. Bir Sonraki Haftaya Köprü

Bu hafta bir sınıfın **kendi içindeki** iki düzeyi ayırt etmeyi öğrendik: sınıf düzeyi ve nesne düzeyi. Hafta 7'den itibaren sınıflar **arasındaki** ilişkilere geçiyoruz. `extends` anahtar sözcüğüyle bir sınıf başka bir sınıfın özelliklerini devralacak; Hafta 5'te öğrendiğimiz yapıcı zinciri `super(...)` çağrısıyla sınıf hiyerarşisine yayılacaktır.

Bu haftaki kavram	Hafta 7'de nasıl genişleyecek?
`static` alan	`static` alanlar **miras alınır** ama ezilemez; alt sınıf üst sınıfın sayacını paylaşır
`static` metot	`static` metotlar ezilmez, **gizlenir** (hiding); çok biçimlilik `static` üyelerde çalışmaz
`private` yapıcı	`private` yapıcılı sınıf miras alınamaz; alt sınıf üst yapıcıyı çağıramaz
`final` sınıf	`final` sınıf `extends` edilemez; `MatematikYardimcisi` bilinçli olarak böyle yazılmıştı
Sınıf yükleme sırası	Kalıtımda üst sınıf **önce** yüklenir; `static` bloklar yukarıdan aşağıya çalışır
`protected`	Hafta 4'te tanımlanmıştı; asıl anlamını Hafta 7'de alt sınıf bağlamında kazanacak

Tablo 6.13 — Hafta 6 kavramlarının Hafta 7'deki karşılıkları

Daha uzun vadede: Bölüm 4.6'daki tekil sezgisi Hafta 14'te **Singleton** deseni olarak; `static` fabrika metotları aynı hafta **Factory** deseni olarak geri dönecektir. `static final` sabitler Hafta 10'da arayüz sabitleri bağlamında; yardımcı sınıflar ise Hafta 13'te `Collections` ve `Arrays` sınıflarıyla yeniden karşımıza çıkacaktır.

9. Haftanın Özeti

- **`static` bir aidiyet bildirimidir:** üye nesneye değil, sınıfa aittir. Kaç nesne üretilirse üretilsin bellekte tek kopyası vardır.
- **Örnek alan nesne başına bir kopyadır.** Birinde yapılan değişiklik diğerlerini etkilemez; `static` alanda ise değişiklik herkes için geçerlidir.
- **`static` metot örnek üyelere erişemez**, çünkü `this`'i yoktur — hangi nesnenin alanı olduğu belirsizdir. Tersi serbesttir.
- **Sınıf yükleme tembeldir.** Sınıf, ilk gerçekten kullanıldığında yüklenir; `static` alan başlatıcıları ve `static` bloklar sınıf başına **yalnızca bir kez** ve yazım sırasına göre çalışır.
- **`static` bloğun en değerli kullanımı kurulum doğrulamasıdır.** Tutarsız bir kurulum, programı sessizce yanlış çalıştırmak yerine erken durdurmalıdır.
- **`static final` sabitler sihirli sayıları ortadan kaldırır.** Ad, belgenin kendisidir; değişiklik tek satırdan yapılır.
- **`final` referansı dondurur, içeriği korumaz.** Dizi ve koleksiyon sabitleri `private` tutulup savunmalı kopya döndürülmelidir.

- **Sayaç deseni benzersizlik güvencesini çağıran koddan sınıfa taşır.** Sipariş, fatura, oturum ve birincil anahtar üretiminin temelidir.
- **Yardımcı sınıf üç kurala uyar:** tüm üyeler `static`, yapıcı `private`, sınıf `final`. Sınama sorusu: "iki nesnesi arasında fark olur muydu?"
- **En tehlikeli `static` hataları derleyicinin sessiz kaldığı hatalardır.** Nesneye özgü bir alanı `static` yapmak derlenir, çalışır ve yanlış sonuç üretir.

→ BİR SONRAKİ HAFTA

Hafta 7 — Miras (Inheritance). `extends` anahtar sözcüğü, alt ve üst sınıf ilişkisi, `super` ile yapıcı zincirinin hiyerarşiye yayılması, metot ezme girişi, `Object` sınıfı, IS-A / HAS-A kararı ve Java'nın çoklu sınıf mirasını neden yasakladığı (elmas problemi).

10. Alıştırmalar

Alıştırmalar üç kümeye ayrılmıştır: kavram soruları (1–4), kod okuma ve hata bulma (5–7), programlama (8–12). Zorluk göstergesi: ●○○ temel, ●●○ orta, ●●● ileri. Derse gelmeden önce `Ogrenci`, `MatematikYardimcisi` ve `StatikBlokOrnegi` sınıflarını gözden geçirmeniz önerilir.

10.1. Kavram Soruları

Alıştırma 1 ●○○

"`static` bir görünürlük belirleyicisi değildir" cümlesini kendi cümlelerinizle açıklayınız. C dilindeki `static` ile farkını, her iki dilden birer kod parçası vererek gösteriniz. Bölüm 5'teki tabloyu dayanak olarak kullanabilirsiniz.

Alıştırma 2 ●●○

Bir `static` metodun örnek alanına erişememesinin gerekçesini `this` üzerinden açıklayınız. Bu kural olmasaydı `System.out.println(ad);` satırı hangi nesnenin `ad` alanını yazdırırdı? Sorunun yanıtlanamaz olduğunu gösteriniz.

Alıştırma 3 ●●○

Sınıf yüklemenin **tembel** olmasının iki somut yararını ve bir olası sakıncasını yazınız. Bir programın 200 sınıfı varsa ve çalışma sırasında yalnızca 30'u kullanılıyorsa, tembel yükleme ne kazandırır?

Alıştırma 4 ●●●

`public static final int[] ESIKLER = {50, 60, 70};` bildiriminin neden gerçek bir sabit olmadığını açıklayınız. Ardından, bu bildirimden yararlanarak bir programın davranışını dışarıdan değiştiren üç satırlık bir kod yazınız ve iki farklı çözüm öneriniz.

10.2. Kod Okuma ve Hata Bulma

Alıştırma 5 ●●○

Aşağıdaki sınıfta **üç ayrı** kusur vardır. Üçünü de bulup nasıl düzelteceğinizi yazınız. Hangisi derleme hatası verir, hangisi sessizce yanlış çalışır?

```java
public class Urun {
    static String ad;                       // (1)
    static double fiyat;
    static int uretilenUrun = 0;

    public Urun(String ad, double fiyat) {
        Urun.ad = ad;
        Urun.fiyat = fiyat;
        uretilenUrun++;
    }

    public static double kdvli() {          // (2)
        return fiyat * 1.20;
    }

    public static void main(String[] args) {
        Urun a = new Urun("Kalem", 10.0);
        Urun b = new Urun("Defter", 25.0);
        System.out.println(a.ad + " " + a.kdvli());   // (3)
        System.out.println(b.ad + " " + b.kdvli());
    }
}
```

Kod 6.19 — Alıştırma 5 için inceleme kodu

Alıştırma 6 ●●●

Aşağıdaki program çalıştırıldığında ekrana ne yazılır? Yanıtınızı **çalıştırmadan önce** yazınız, sonra doğrulayınız. Sıra beklediğinizden farklı çıktıysa nedenini Bölüm 4.1'e dayanarak açıklayınız.

```java
public class Yukleme {
    static int a = yaz("A");
    static { yaz("B"); }
    int c = yaz("C");
    { yaz("D"); }
    static int e = yaz("E");

    Yukleme() { yaz("F"); }

    static int yaz(String t) { System.out.print(t + " "); return 0; }

    public static void main(String[] args) {
        System.out.print("BASLA ");
        new Yukleme();
```

```
        System.out.print("| ");
        new Yukleme();
        System.out.print("| " + Yukleme.a);
    }
}
```

Kod 6.20 — Alıştırma 6 için inceleme kodu

Alıştırma 7 ●●○

Aşağıdaki `Sepet` sınıfı derlenir ve çalışır; ancak iki müşterinin sepeti karışır. Sorunu bulunuz, sorunu **gösteren** dört satırlık bir sürücü program yazınız ve düzeltiniz. Düzeltme sonrasında hangi alanın `static` kalması gerektiğini gerekçelendiriniz.

```
public class Sepet {
    private static double toplam = 0.0;
    private static int urunSayisi = 0;
    private static int acilanSepet = 0;
    private final String musteri;

    public Sepet(String musteri) {
        this.musteri = musteri;
        acilanSepet++;
    }

    public void ekle(double fiyat) {
        toplam += fiyat;
        urunSayisi++;
    }

    public double tutar() { return toplam; }
}
```

Kod 6.21 — Alıştırma 7 için inceleme kodu

10.3. Programlama Alıştırmaları

Alıştırma 8 ●○○

`Musteri` adında bir sınıf yazınız. Her müşteriye `MUS-0001` biçiminde otomatik bir numara verilsin. Sınıf ayrıca üretilen toplam müşteri sayısını ve tüm müşterilerin toplam bakiyesini `static` alanlarda tutsun. `static double ortalamaBakiye()` metodu yazınız ve hiç müşteri yokken sıfıra bölme oluşmamasını sağlayınız.

İpucu: `Kitap.java` dosyasındaki `numaraUret()` metodunu ve `ortalamaKredi()` içindeki sıfır denetimini örnek alabilirsiniz.

Alıştırma 9 ●●○

`MetinAraclari` adında bir yardımcı sınıf yazınız. Üç kurala tam olarak uysun. Şu `static` metotları sunsun: `tersCevir(String)`, `palindromMu(String)`, `kelimeSay(String)`, `basHarfleriBuyut(String)`. Sınıf ayrıca kaç kez çağrıldığını `private static` bir sayaçta izlesin ve bunu `static` bir getter ile sunsun.

İpucu: `BirimCevirici.java` dosyasındaki `say()` deseni tam olarak bu işi yapar. `null` girdileri `IllegalArgumentException` ile

reddediniz.

Alıştırma 10 ●●○

`Yapilandirma` adında bir sınıf yazınız. Sınıf, bir `static` blok içinde şu üç kuralı doğrulasın: (i) azami kullanıcı sayısı pozitif olmalı, (ii) zaman aşımı 1–300 saniye aralığında olmalı, (iii) günlük düzeyi geçerli bir değer olmalı. Kurallardan biri bozulduğunda sınıfın hiç yüklenmediğini **çalıştırarak** gösteriniz ve alınan hatanın tam adını raporunuza yazınız.

İpucu: `SinavKurallari.java` dosyasındaki `static` bloğu örnek alınız. Bozuk sürümü ayrı bir dosyada saklayıp iki çıktıyı yan yana koyunuz.

Alıştırma 11 ●●●

`Envanter` adında bir sınıf yazınız. Sınıfta `private static final String[] KATEGORILER` bulunsun. Önce bu diziyi `public` yaparak dışarıdan bozulabildiğini gösteriniz; ardından savunmalı kopya ile koruyunuz. İki sürümün çıktısını karşılaştıran kısa bir rapor yazınız. Raporda `clone()` çağrısının **iki boyutlu** dizilerde neden yetmediğini de açıklayınız.

İpucu: Bölüm 4.5'teki `SinavKurallari.esikler()` metodunu ve Hafta 5, Bölüm 4.2'deki “nesne dizisi” satırını yeniden okuyunuz.

Alıştırma 12 ●●○

Bir `Sayac` sınıfı yazarak `static` alan ile örnek alanın farkını **sayısal olarak** gösteriniz: sınıfta bir `static int paylasilan` ve bir `int kendi` alanı olsun; her ikisini de artıran bir `artir()` metodu bulunsun. Beş nesne üretip her birinde `artir()` metodunu farklı sayıda çağırınız. Sonuçları bir tablo hâlinde yazdırınız ve `paylasilan` alanının neden tüm çağrıların toplamına eşit olduğunu açıklayınız.

İpucu: Çıktı biçimlendirmesi için `System.out.printf` kullanınız; `OgrenciDemo.java` dosyasının 6. bölümü örnek bir tablo yazdırma biçimi gösterir.

11. Öz Değerlendirme

Aşağıdaki maddelerin tamamını işaretleyebiliyorsanız bu haftanın kazanımlarına ulaşmışsınız demektir. İşaretleyemediğiniz her madde için karşısındaki bölüme dönünüz.

Öz Değerlendirme Kontrol Listesi

- ☐ `static` alan ile örnek alanın bellekteki yerleşimini çizebiliyorum. *(Bölüm 3.1, Şekil 6.2)*
- ☐ Bir üyenin `static` mi örnek üyesi mi olacağına gerekçeli karar verebiliyorum. *(Bölüm 4.7, Şekil 6.9)*
- ☐ `static` metodun neden örnek alanlara erişemediğini `this` üzerinden açıklayabiliyorum. *(Bölüm 3.3)*
- ☐ `Cannot make a static reference to the non-static field` hatasını okuyup düzeltebiliyorum. *(Bölüm 6.1, 6.7)*
- ☐ Sınıf yüklemenin tembel olduğunu ve `static` blokların bir kez çalıştığını çıktı üzerinde gösterebiliyorum. *(Bölüm 4.1)*
- ☐ `static` blokla kurulum doğrulaması yazabiliyorum. *(Bölüm 4.2)*
- ☐ Sihirli sayıları `static final` sabitlere dönüştürebiliyorum. *(Bölüm 4.4)*
- ☐ `final`'ın referans türlerde neyi korumadığını açıklayıp savunmalı kopya yazabiliyorum. *(Bölüm 4.5, Şekil 6.7)*
- ☐ Otomatik kimlik üreten sayaç desenini uygulayabiliyorum. *(Bölüm 2.2, Şekil 6.4)*
- ☐ Üç kurala uyan bir yardımcı sınıf yazabiliyor ve ne zaman uygun olduğunu değerlendirebiliyorum. *(Bölüm 4.3)*

12. Kaynaklar

12.1. Ders Kitapları

Kaynak	İlgili bölüm	Not
Deitel & Deitel, *Java How to Program* (11. baskı)	Bölüm 8.11 — `static` Class Members; Bölüm 8.12 — `final` Instance Variables	Temel düzey, bol örnekli
Horstmann, *Core Java Volume I* (12. baskı)	Bölüm 4.4 — Static Fields and Methods (static constants, factory methods, the `main` method)	Sınıf yükleme ve sabitler için en açıklayıcı anlatım
Schildt, *Java: The Complete Reference* (12. baskı)	Bölüm 7 — Understanding `static`, Introducing `final`	Başvuru kaynağı
Bloch, *Effective Java* (3. baskı)	Madde 4 (yardımcı sınıfta private yapıcı), 17 (değişmezlik), 50 (savunmalı kopya)	Haftanın tasarım felsefesi

Tablo 6.14 — Bu hafta için önerilen okumalar

12.2. Çevrimiçi Kaynaklar

- **Oracle Java Tutorials — Understanding Class Members:** `static` alan ve metotların temel anlatımı, `this` ile ilişkisi.
- **Oracle Java Tutorials — Initializing Fields:** `static` başlatma blokları ve alan başlatıcılarının çalışma sırası.
- **Java Language Specification, §8.3.1.1 — `static` Fields:** sınıf değişkenlerinin biçimsel tanımı (ileri düzey okuma).
- **Java Language Specification, §12.4 — Initialization of Classes and Interfaces:** sınıfın hangi olaylarda yükleneceğinin tam listesi.
- **Java Language Specification, §4.12.4 — `final` Variables:** derleme zamanı sabiti (constant variable) tanımı ve gömme kuralı.

12.3. İletişim ve Destek

Konu	Sorumlu	Kanal
Teori, kavram soruları	Dr. Öğr. Üyesi Rıfat BENVENİSTE	Ders saati sonrası · e-posta
Laboratuvar, kod hataları	Arş. Gör. Betül MUMCU	Laboratuvar saatleri · e-posta
Teslim ve puanlama	Arş. Gör. Betül MUMCU	Ders yönetim sistemi
Ortam kurulumu (JDK, VS Code)	Laboratuvar sorumlusu	Hafta 1 föyü, Görev 0

Tablo 6.15 — Soru ve destek için başvuru noktaları

> **ⓘ BİLGİ**
>
> Bu haftanın tüm Java kaynak dosyaları `Hafta_06/Java_Kodlari/` klasöründedir: ders örnekleri `ders/`, laboratuvar çözümleri ise laboratuvar oturumundan sonra `laboratuvar/` altında paylaşılacaktır. Kod bloklarındaki satır numaraları, listenin kendi içinde 1'den başlar; dosyadaki mutlak numaralara değil.